Project – High Level Design

On

Construction Planning Assistant Agent

Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	SHIWANI PATHAK	EN22CS301921
2.	SHRISHTI SAHU	EN22CS301936
3.	SHREYAS	EN22CS301934
4.	SANSKAR JAIN	EN22CS301868
5	RAJVEER SINGH CHANGAL	EN23CS3T1009

Group Name: Group 03D8

Project Number: AAI-38

Industry Mentor Name:

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2025-2026

Table of Contents

1. Introduction

1.1 Scope of the Document

This document provides the **High Level Design (HLD)** of the *Construction Planning Assistant Agent*. It describes the **overall system architecture**, **components**, **process flow**, **data handling**, **interfaces**, and **non-functional requirements**.

The document is intended to serve as a **technical reference** for development, testing, evaluation, and future enhancements.

1.2 Intended Audience

- Faculty evaluators
- Industry and academic mentors
- Project reviewers
- Developers and maintainers
- Students involved in future enhancements

1.3 System Overview

The Construction Planning Assistant Agent is an **AI-driven planning system** that accepts a **high-level construction goal** and autonomously:

- Decomposes it into tasks
- Identifies task dependencies
- Validates resource availability
- Estimates time and cost
- Optimizes task order
- Generates a final execution schedule
- Performs risk analysis
- Visualizes the plan via a web interface

The system uses a **multi-agent architecture**, **rule-based fallback planning**, and **LLM-based intelligence (Groq)**.

2. System Design

2.1 Application Design

The application follows a **Client–Server architecture**:

- **Frontend:**
  - HTML, CSS, JavaScript
  - Displays AI reasoning steps, schedules, risk, and Gantt chart

- **Backend:**
  - Python + Flask
  - Hosts planner agents, business logic, and APIs
- **AI Layer:**
  - Groq LLM for intelligent task decomposition
  - Offline fallback planner

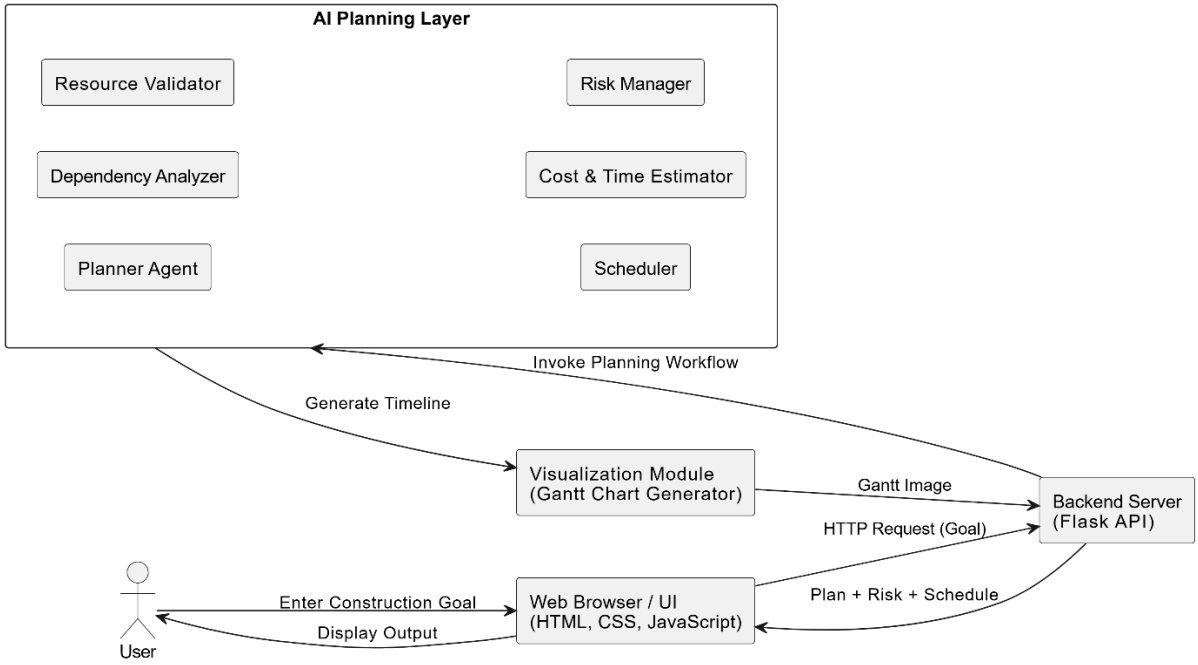

Fig. High Level Architecture Block Diagram

2.2.1 Process Flow

1. User enters a construction goal via UI
2. Goal is sent to backend API
3. Planner Agent decomposes the goal
4. Dependency Agent builds dependency graph
5. Resource Agent validates availability
6. Cost & Time Estimator computes estimates
7. Scheduler optimizes task order
8. Risk Manager evaluates delays
9. Gantt chart is generated
10. Complete plan is returned to frontend

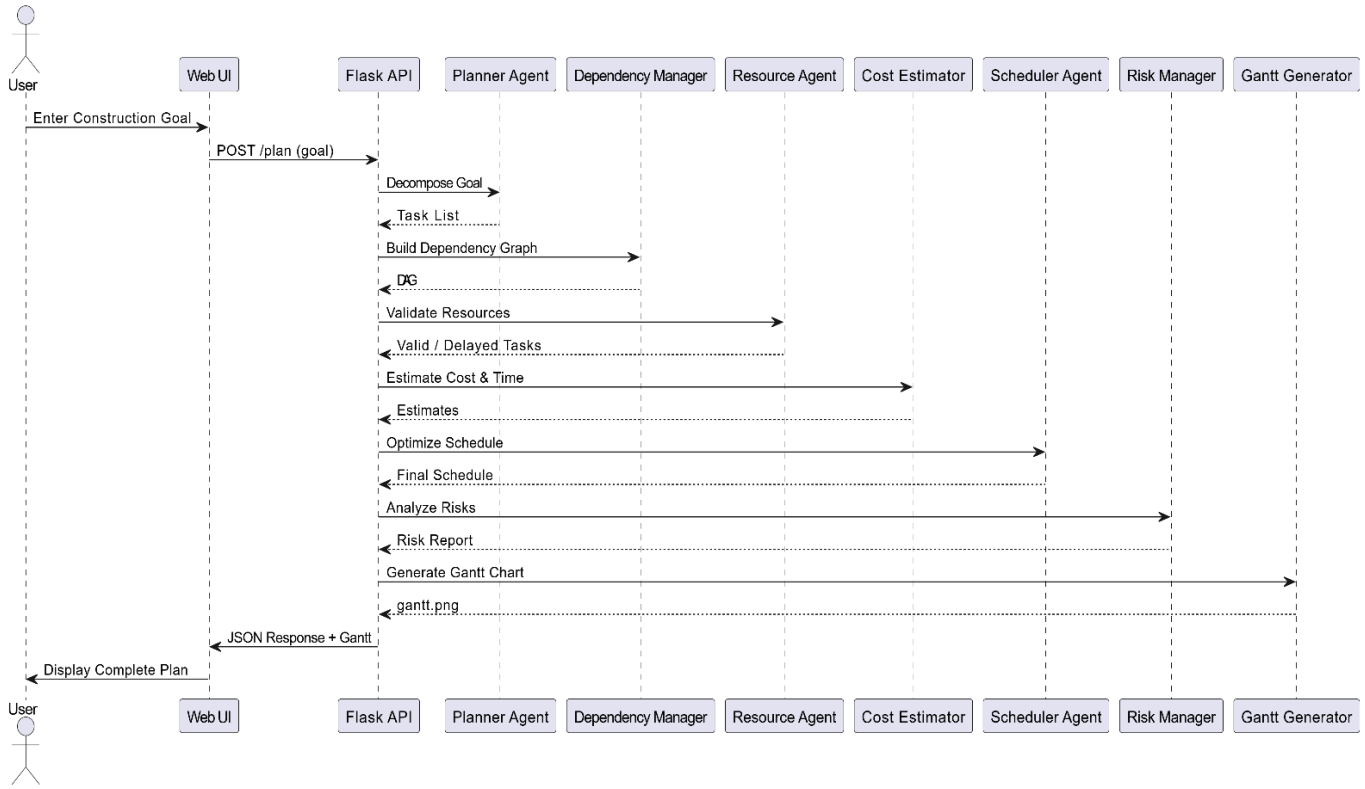

Fig. Sequence Diagram

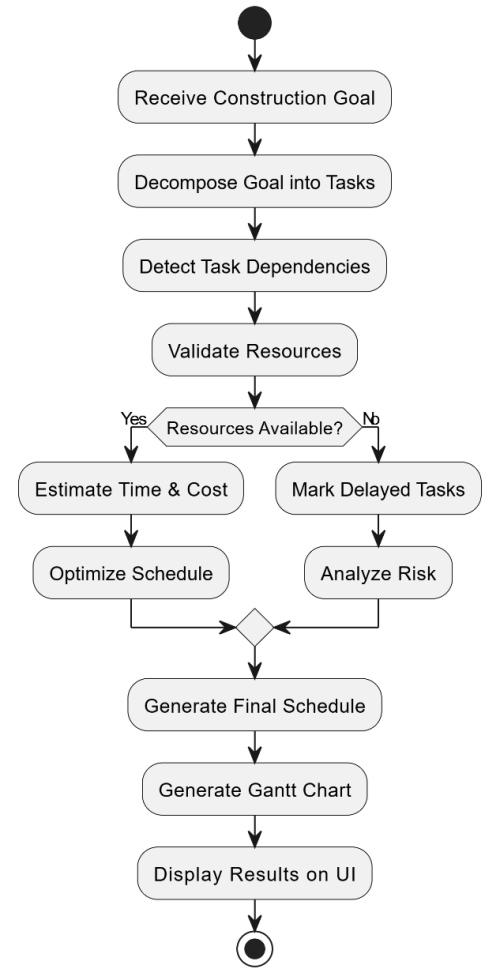

Fig. Activity Diagram

2.3 Information Flow

- Input: User Goal
- Processing: Task data, dependency graph, estimates
- Output: Optimized schedule, risks, visual charts

Data flows **unidirectionally** from UI → Backend → UI ensuring clarity and traceability.

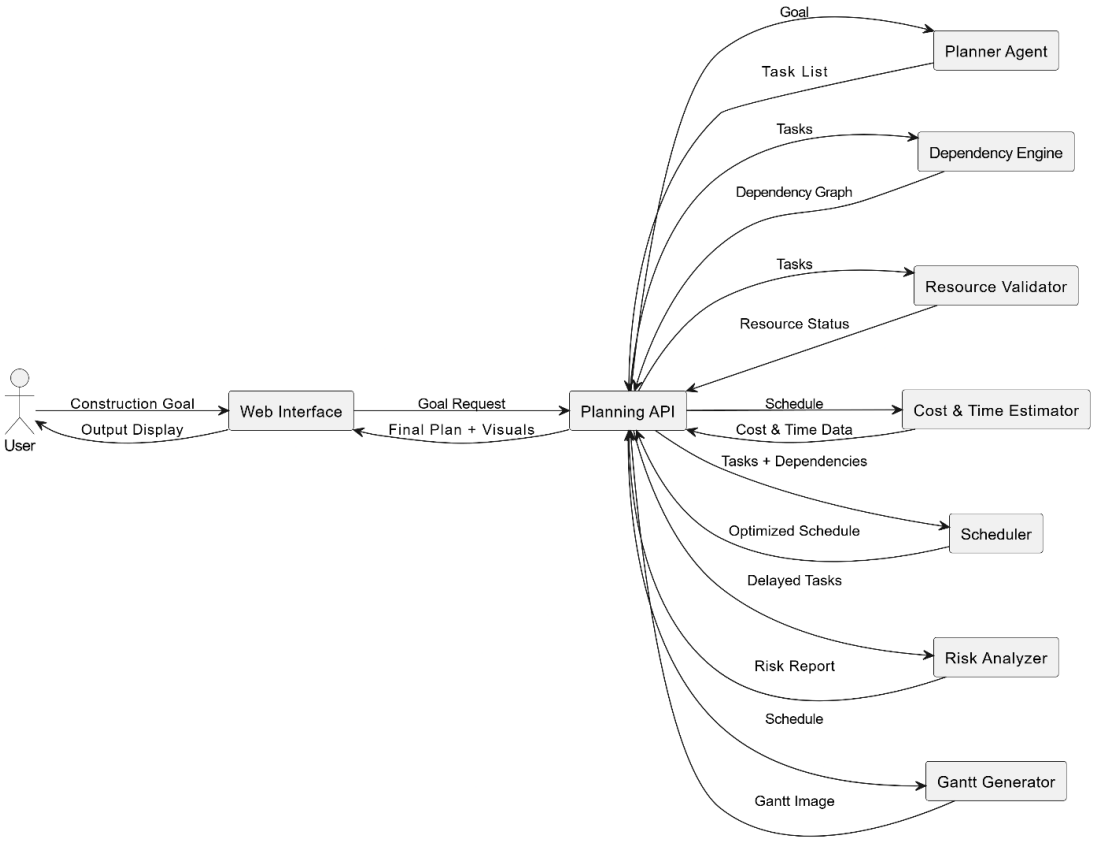

Fig. Dataflow Diagram

2.4 Components Design

Component	Description
Planner Agent	Breaks goal into tasks
Dependency Graph Module	Builds DAG
Resource Agent	Validates availability
Cost Estimator	Estimates time and cost
Scheduler Agent	Optimizes task order
Risk Manager	Detects project risks
Gantt Generator	Visualizes schedule
Web UI	Displays all outputs

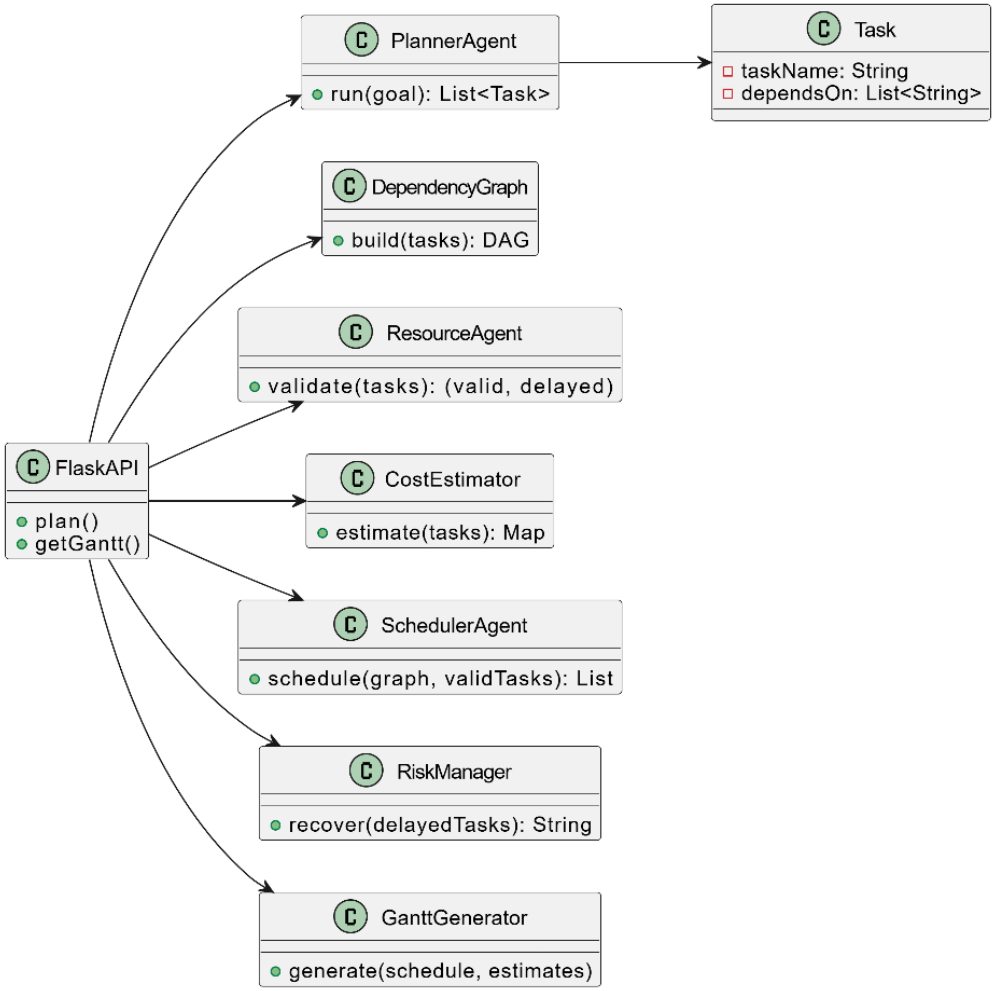

Fig. Class Diagram

2.5 Key Design Considerations

- Modular and scalable design
- Fault tolerance using fallback planner
- API-driven architecture
- Clear visualization for evaluation
- Minimal dependency on paid APIs

2.6 API Catalogue

API	Method	Description
/plan	POST	Generates full construction plan
/gantt.png	GET	Returns Gantt chart image

3. Data Design

3.1 Data Model

The system uses **in-memory structured data**:

- Task Object
- Dependency Graph
- Schedule List
- Cost & Time Dictionary

Persistent storage is not required for the current scope.

3.2 Data Access Mechanism

- JSON-based API communication
- Internal Python data structures
- No external database dependency

3.3 Data Retention Policies

- Data is retained only for the session
- No personal data stored
- Complies with academic project standards

3.4 Data Migration

Not applicable for current academic scope.

4. Interfaces

User Interface

- Responsive web dashboard
- Displays:
    - Goal received
    - Task decomposition
    - Dependencies
    - Resource validation
    - Time & cost
    - Optimization
    - Final schedule
    - Risk analysis

- Gantt chart

5. State and Session Management

- Stateless backend design
- Each request handled independently
- No server-side session storage

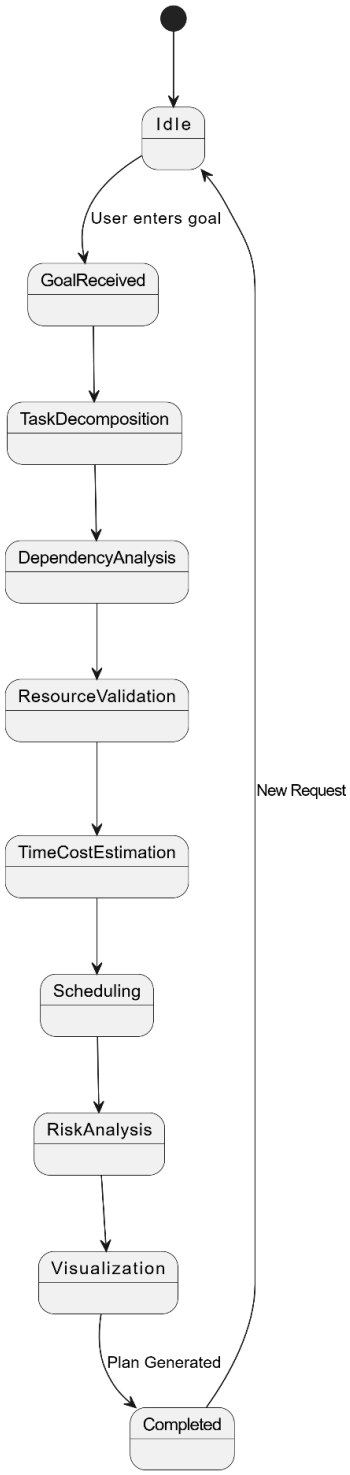

Fig. State Diagram

6. Caching

- Browser caching for static assets
- Gantt chart refreshed using timestamp-based invalidation

7. Non-Functional Requirements

7.1 Security Aspects

- CORS-enabled controlled access
- No authentication required (academic demo)
- No sensitive data handling

7.2 Performance Aspects

- Lightweight APIs
- Fast response (< 2 seconds typical)
- Efficient DAG-based scheduling

8. References

1. Medicaps University – Datagami Skill Based Course, *Agentic AI Project Guidelines*.
2. Russell, S., & Norvig, P., **Artificial Intelligence: A Modern Approach**, Pearson Education.
3. Malik, S., & Martens, C., *AI Planning and Scheduling Systems – Concepts and Applications*.
4. Flask Documentation, *Flask Web Framework*, Official Docs.
5. Groq Documentation, *LLM Inference and API Usage*, Official Developer Guide.
6. UML Documentation, *Unified Modeling Language – Diagrams and Notations*.
7. PMI (Project Management Institute), *Project Scheduling and Risk Management Concepts*.
8. GeeksforGeeks, *Directed Acyclic Graphs (DAG) and Scheduling Algorithms*.